

1. Architecture du Frontend

Le frontend est développé en **React** avec **Vite** comme outil de construction pour garantir une rapidité et une performance optimales. Il permet à l'utilisateur d'interagir avec le système, de créer des DPP, de consulter les résultats d'audit, et plus encore.

Composants Clés :

- **LandingPage** :
 - Page d'accueil dynamique avec un **héro animé** qui met en avant les **SDG** (Sustainable Development Goals), l'impact de la plateforme, et les logos des partenaires comme **Yonk AI Leaders** et **Bicep**.
 - Un **chatbot** pour interagir avec les utilisateurs.
- **SignInForm** :
 - Formulaire de connexion pour que les utilisateurs puissent accéder à leurs DPPs.
 - Stockage du **token JWT** dans **localStorage** pour la gestion de session.
- **SignUpForm** :
 - Formulaire d'inscription pour la création de nouveaux utilisateurs.
- **UserDashboard** :
 - Interface utilisateur pour voir les **DPPs** créés par l'utilisateur, avec un tableau de bord affichant les **comptes** et les **DPPs récents**.
- **DppForm** :
 - Formulaire permettant aux utilisateurs de soumettre les détails pour **créer un DPP**.
- **AuditResults** :
 - Permet d'uploader un fichier DPP pour **l'audit**, en attendant les résultats de conformité générés par l'IA.
- **ProtectedRoute** :

- Composant pour protéger les pages, s'assurant que l'utilisateur est authentifié avant d'accéder à certaines ressources sensibles.

Pages du Frontend :

- **/index** : Page d'accueil (Landing Page) avec la section héro dynamique.
- **/signin** : Page de connexion.
- **/signup** : Page d'inscription.
- **/user-dashboard** : Page qui présente les DPPs récents de l'utilisateur.
- **/dpp-create** : Page pour la création d'un nouveau DPP.
- **/audit** : Page pour télécharger un DPP et afficher les résultats de l'audit.

Communication avec le Backend :

- **api.js** : Fichier utilitaire pour la gestion des appels API (authentification, création de DPP, téléchargement d'audit).
- **auth.js** : Pour vérifier si l'utilisateur est authentifié (via **localStorage**).

=====

2. Architecture du Backend

Le backend est construit avec **FastAPI** (framework Python), qui est conçu pour offrir des performances élevées, des temps de réponse rapides, et une gestion facile de l'authentification via **JWT**. Le backend gère la création des DPP, l'audit des fichiers, la gestion des utilisateurs, ainsi que les fonctionnalités d'administration.

Composants Clés :

- **main.py** :
 - Point d'entrée de l'application backend. Il initialise l'application FastAPI, configure **CORS** et gère les routes pour les **authentifications**, **DPPs**, et **audits**.
- **security.py** :

- Gère le **hachage des mots de passe** via **bcrypt** et l'émission des **tokens JWT** pour l'authentification des utilisateurs.
- **admin.py** :
 - Protège les routes d'administration via un jeton **X-Admin-Token**, garantissant que seules les personnes autorisées peuvent accéder à ces routes.
- **session.py** :
 - Permet de gérer les **sessions SQLAlchemy** pour interagir avec la base de données.
- **init_db.py** :
 - Initialise la base de données en créant les tables à partir des métadonnées SQLAlchemy.
- **auth.py** :
 - Fournit les **routes d'inscription** et de **connexion** des utilisateurs. Lors de la connexion, un **token JWT** est généré et renvoyé à l'utilisateur.
- **dpp_creation.py** :
 - Gère la **création** et l'**affichage des DPPs** pour les utilisateurs. Les données sont enregistrées dans la base de données avec un payload JSONB.
- **audit.py** :
 - Permet d'**uploader un fichier DPP** pour l'audit, en utilisant des modèles d'IA pour le scoring et l'analyse (actuellement avec des modèles placeholders).
- **users.py** :
 - Fournit des informations sur le profil de l'utilisateur et la gestion de leur tableau de bord.
- **admin.py** :
 - Gère la liste des utilisateurs pour l'administration (seulement accessible par les admins).

Placeholders IA :

- **compliance_model.py** :
 - Fonctionnalité de scoring de conformité simulée, en attente de l'intégration des vrais modèles d'IA pour la conformité.
- **transformer_model.py** :
 - Modèle placeholder pour enrichir les DPPs avec des insights supplémentaires via l'IA.

Base de Données :

- **PostgreSQL** : Utilisé pour stocker les informations sur les utilisateurs, les DPPs et les résultats d'audit.
- **SQLAlchemy** : ORM pour interagir avec la base de données de manière simple et flexible.

Authentification et Sécurité :

- **JWT (JSON Web Tokens)** : Pour authentifier les utilisateurs et sécuriser l'accès aux différentes routes.
- **bcrypt** : Pour sécuriser le stockage des mots de passe.

=====

3. Structure du Projet

Frontend (React + Vite)

```
/frontend
/src
  /components    # Composants UI (Header, Footer, DppForm, etc.)
  /pages         # Pages React pour chaque route (Landing, SignIn, etc.)
  /styles        # Feuilles de style CSS
  /utils         # Fichiers utilitaires (api.js, auth.js)
/public
  index.html     # Fichier principal
  vite.config.js # Configuration Vite
  package.json   # Dépendances et scripts frontend
```

Backend (FastAPI + SQLAlchemy)

```
/backend
/app
  /api/v1
    /auth.py      # Routes d'authentification
    /dpp_creation.py # Routes de création et affichage des DPPs
    /audit.py     # Routes d'audit
    /users.py     # Routes utilisateurs
    /admin.py     # Routes administrateurs
  /core
    /security.py  # Sécurisation des mots de passe et des tokens
    /config.py   # Configuration du projet
    /admin.py    # Gestion des droits d'accès admin
  /db
    /models.py   # Modèles SQLAlchemy
    /session.py  # Gestion des sessions DB
    /init_db.py  # Initialisation de la DB
  /ai
    /models      # Modèles d'IA (compliance_model, transformer_model)
  /scripts
    /seed_user.py # Script pour créer un utilisateur de test
  /tests
    /test_dpp.py  # Tests unitaires (par exemple, test de la santé)
requirements.txt  # Dépendances Python
package.json     # Fichier de configuration backend
```

=====

- **MCP ⇒ API (Oussama)**
- **Des sources de données DPPs ⇒ Traitement de donnée ⇒ Audit (Kiana - Bryan)**
- **Integration du LLM (Kiana - Bryan)**
- **Déploiement du projet (Oussama - Bryan)**
- **Génération de QR Code pour les DPP (Après ...)**
- **Benchmarking (Kiana)**
- **Sécurité + Gouvernance de données (Oussama ...)**
- **Scoring and transformers...**